



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

International Training & Cooperation Institute

Introduction to Blockchain Technology

Lesson 8 – Ethereum Consensus, Validation, and Execution



Ethereum Consensus, Validation, and Execution



- 1. Ethereum Proof of Stake Architecture**
- 2. Validators: Entry, Selection, and Roles**
- 3. Block Proposal and Voting**
- 4. Fork Choice and Finality**
- 5. Execution and Validator Rewards**
- 6. Summary**
- 7. Discussion**
- 8. Introduction to Web3 Wallets & MetaMask**

1. Ethereum Proof of Stake Architecture

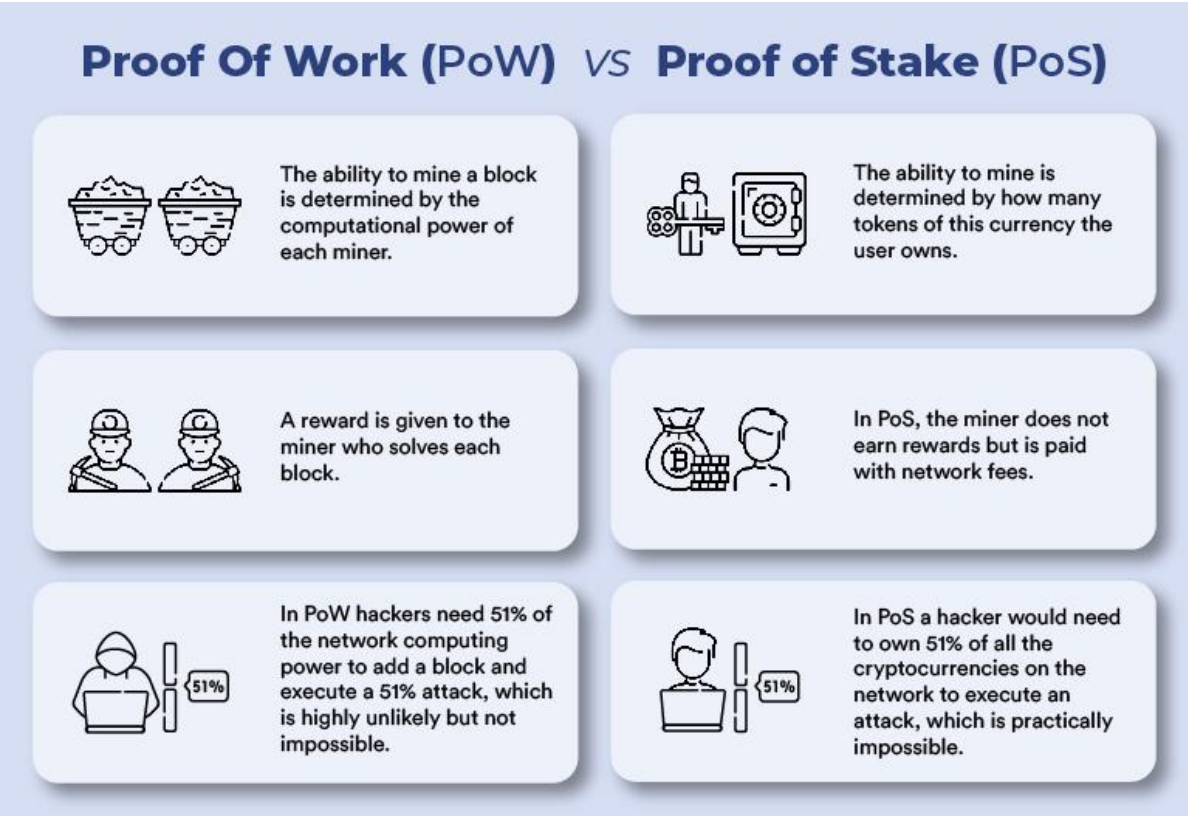
Ethereum no longer relies on Proof of Work mining.

Mining drawbacks:

- ❖ High energy consumption
- ❖ Hardware centralization
- ❖ Slow finality

Ethereum: Proof of Stake

- ❖ Replace energy cost with economic stake
- ❖ Achieve faster and more predictable consensus
- ❖ Reduce environmental impact

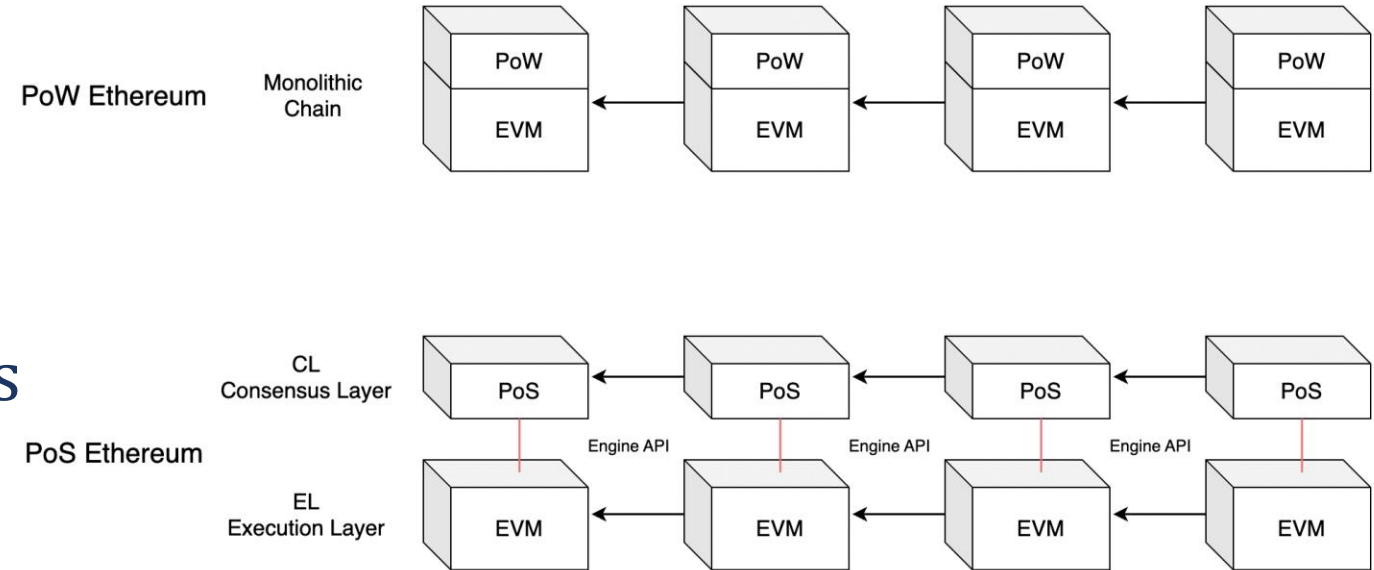


Ethereum transitioned from Proof of Work to Proof of Stake in 2022 via a major protocol upgrade known as *The Merge*

1. Ethereum Proof of Stake Architecture



Ethereum consensus involves two tightly connected layers:



Execution Layer (EL): transactions, smart contracts, EVM

Consensus Layer (CL): validator selection, voting, finality

Validators:

- ❖ Lock **32 ETH** as stake
- ❖ Run both **EL + CL** clients
- ❖ Participate in proposing and voting on blocks

Consensus is achieved by agreement on **execution results**

2. Validators: Entry, Selection, and Roles

Ethereum divides time into:

- ❖ **Slots:** 12 seconds
- ❖ **Epochs:** 32 slots

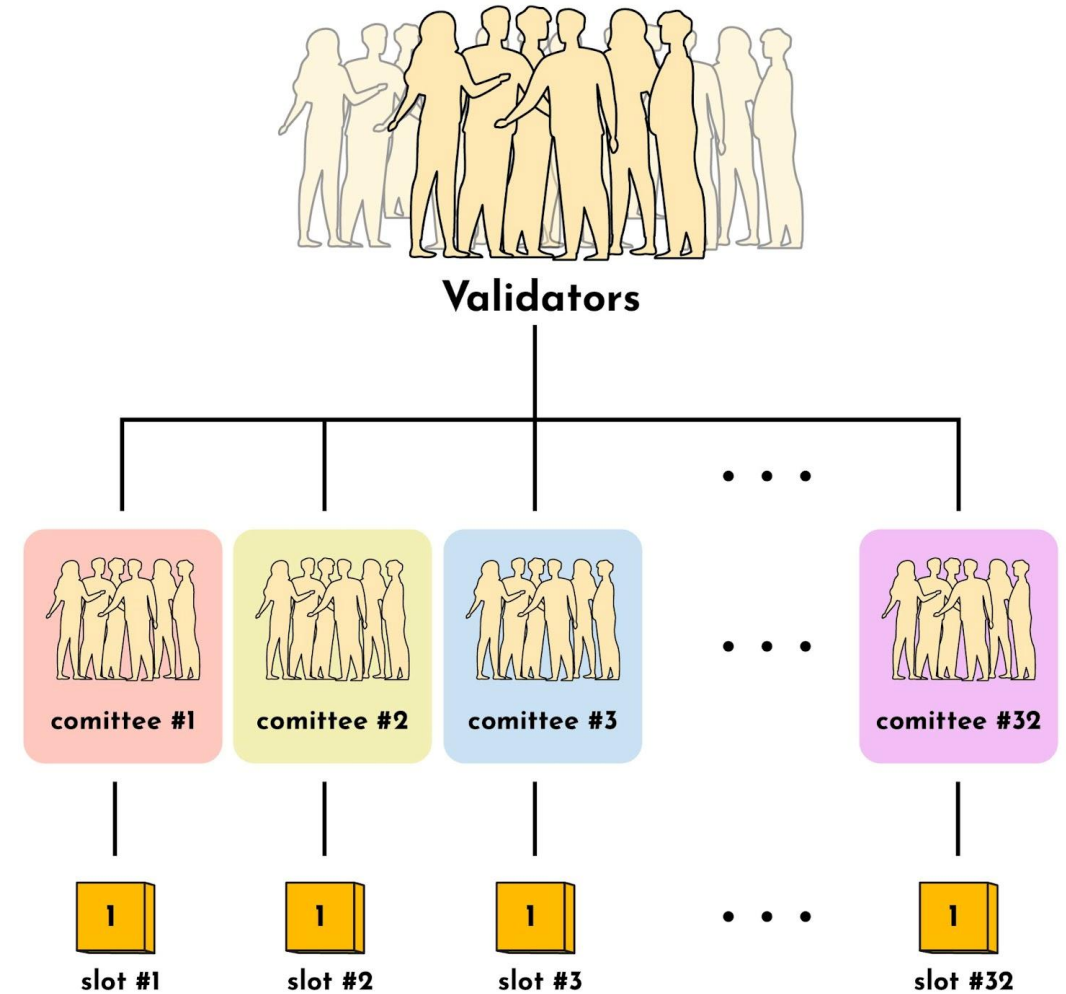
For each slot:

- ❖ The protocol **computes exactly one validator**
- ❖ This validator is the only one allowed to propose a block

2. Validators: Entry, Selection, and Roles

Not everyone vote on every block:

- ❖ Validators are grouped into **committees**
- ❖ One validator becomes **block proposer**
- ❖ Others become **attesters (voters)**



2. Validators: Entry, Selection, and Roles

How Validator selection works :

- ❖ Uses RANDAO + Verifiable Delay Function (VDF)
- ❖ Selection is:
 - Deterministic
 - Unpredictable in advance
 - Computed independently by all nodes

Example: *Out of 500,000 validators:*

1 proposer per slot/ ~128 validators per committee

Committee votes on proposer's block. Each validator is selected on average once every ~6 days to propose a block, but attests much more frequently

Q: Can a validator choose to propose a block anytime?

2. Validators: Entry, Selection, and Roles

Validator Responsibilities

Once active, a validator must:

- ❖ Run a full Ethereum node
- ❖ Propose blocks when selected
- ❖ Verify blocks proposed by others
- ❖ Vote honestly and remain online

Validators that fail:

- ❖ Lose rewards
- ❖ May be penalized or removed

3. Block Proposal and Voting

Voting (Attestation):

- ❖ A cryptographic signature from a validator
- ❖ Supporting a specific block on a specific chain

Validators vote **only if**:

- ❖ Block structure is valid
- ❖ Transactions are valid
- ❖ EVM execution produces correct state
- ❖ Block follows consensus rules

3. Block Proposal and Voting

Why Voting Is Needed

Unlike PoW:

- ❖ There is no hash race
- ❖ Multiple valid blocks may exist briefly

⇒ Ethereum uses **explicit voting** to choose the canonical block.

Only one block can become finalized because global state must remain consistent across all nodes.

3. Block Proposal and Voting

Block Proposal Process

When selected, a validator:

- ❖ Selects pending transactions
- ❖ Executes them locally using EVM
- ❖ Computes the new global state
- ❖ Creates a block
- ❖ Broadcasts the block to the network

At this point: The block is valid, but **not yet final**.

3. Block Proposal and Voting

Timestamp:

- ❖ Allowed within a **range**
- ❖ Not a strict exact time

Only one proposer is selected on a slot

However, small time differences are acceptable => Timestamp alone does NOT immediately invalidate a block

Example: *At Slot 100, Validator A proposes Block A.*

Some validators receive Block A late

Another block B propagates faster

Temporary fork appears

=> Voting resolves it

4. Fork Choice and Finality

Fork Choice Rule

The chain with the strongest recent validator support

- ❖ Votes are stake-weighted
- ❖ Validators follow majority support
- ❖ Forks are usually short-lived

4. Fork Choice and Finality

Finality in Ethereum

- ❖ $\geq 2/3$ of staked ETH consistently votes
- ❖ Over two consecutive epoch

After finality:

- ❖ Reverting blocks requires massive ETH loss
- ❖ Practically irreversible

Finality provides stronger guarantees than PoW confirmations.

4. Fork Choice and Finality

Ethereum Forks and Protocol Evolution

Ethereum has experienced **intentional forks** for upgrades

Major Fork events:

- ❖ **The DAO Fork (2016)**: governance decision
- ❖ **The Merge (2022)**: PoW → PoS

Intentional Forks occur when:

- ❖ Protocol rules change
- ❖ Majority of validators upgrade

Example: *Nodes that did not upgrade after **The Merge** stayed on PoW chain
=> Result: Ethereum (ETH) vs Ethereum Classic (ETC)*

5. Execution and Validator Rewards

EVM

- ❖ Stack-based instructions
- ❖ Deterministic state transitions

Execution elements:

- ❖ Stack
- ❖ Memory (temporary)
- ❖ Storage (persistent)

Q: Why is storage much more expensive than memory?

5. Execution and Validator Rewards

EVM Opcode

Smart Contract code is **compiled** into EVM **bytecode**, which is executed instruction by instruction.

EVM is a **stack-based machine**:

- ❖ No variables
- ❖ Operates on a **stack** (Last-In-First-Out)

Example

<i>uint x = 5;</i>	<i>Becomes:</i>	<i>PUSH 5</i>
<i>x = x + 1;</i>		<i>PUSH 1</i>
		<i>ADD</i>
		<i>STORE</i>

Each opcode has **fixed gas cost**.

5. Execution and Validator Rewards

Why Ethereum Charges for Execution

Ethereum supports:

- ❖ Loops
- ❖ Storage writes
- ❖ Nested contract calls

Without cost:

- ❖ Infinite execution
- ❖ Resource abuse

Gas ensures fair usage and network protection.

Example: *Writing to storage costs ~20,000 gas, while reading costs ~2,100 gas => smart contract design must minimize storage writes.*

5. Execution and Validator Rewards

What is Gas?

- ❖ **Gas** measures computational work in Ethereum
- ❖ Each EVM operation consumes a fixed amount of gas
- ❖ Total gas used = sum of all executed operations

Gas Price Components: A transaction fee is split into two parts:

❖ **Base Fee**

- Set by the protocol
- Automatically adjusted per block
- **Burned** (removed from circulation)

❖ **Priority Fee (Tip)**

- Chosen by the user
- Paid directly to the block proposer (validator)

Total transaction fee = gas used $\times$ (base fee + priority fee)

5. Execution and Validator Rewards

Validator rewards:

❖ Issuance (new ETH)

- Minted by protocol
- Depends on total ETH staked

❖ Priority fees

- Paid by users
- Given to block proposer

❖ Penalties (negative rewards)

- Offline → missed rewards
- Malicious → slashing

Notes:

- ❖ Base fee is burned, not paid to validators
- ❖ Issuance is dynamic, not halving-based

5. Execution and Validator Rewards

Units: Gwei

- ❖ 1 ETH = 1,000,000,000 gwei
- ❖ Gas prices are measured in **gwei**
- ❖ Gas amount itself is **unitless**

Example: *In a transaction: Gas used = 80,000 / Base fee = 15 gwei / Priority fee = 3 gwei*

ETH burned: $80,000 \times 15 \text{ gwei} = 1,200,000 \text{ gwei}$

ETH paid to validator: $80,000 \times 3 \text{ gwei} = 240,000 \text{ gwei}$

Validators do **not** receive the full transaction fee.

Base fees reduce ETH supply, while priority fees incentivize block production.

Q: Why burn the base fee instead of paying it to validators?

6. Summary

- ❖ Ethereum replaces mining with validator-based consensus.
- ❖ Validators are selected using protocol randomness, not competition.
- ❖ EVM execution defines valid state transitions and gas cost.
- ❖ Rewards and penalties economically enforce honest behavior.
- ❖ Temporary forks are resolved through stake-weighted voting and finality

7. Discussion

1. A transaction uses 60,000 gas.

Base fee = 16 gwei

Priority fee = 2 gwei

Calculate:

ETH burned

ETH paid to validator

2. Why does Ethereum allow temporary forks but enforce strong finality later?

8. Introduction to Web3 Wallets & MetaMask

Tools for Smart Contract Development

- ❖ **Language:** Solidity
- ❖ **IDE:** Remix (remix.ethereum.org)
- ❖ **Development Environment:** Hardhat / Foundry / Truffle
- ❖ **Wallet:** MetaMask
- ❖ **Network:** Ethereum Testnets

Skills to prepare:

- ❖ Reading execution flow
- ❖ Estimating gas cost
- ❖ Understanding transaction lifecycle



8. Introduction to Web3 Wallets & MetaMask

Why We Need a Wallet - Web3 Identity (EOA)

Wallet (MetaMask as an example):

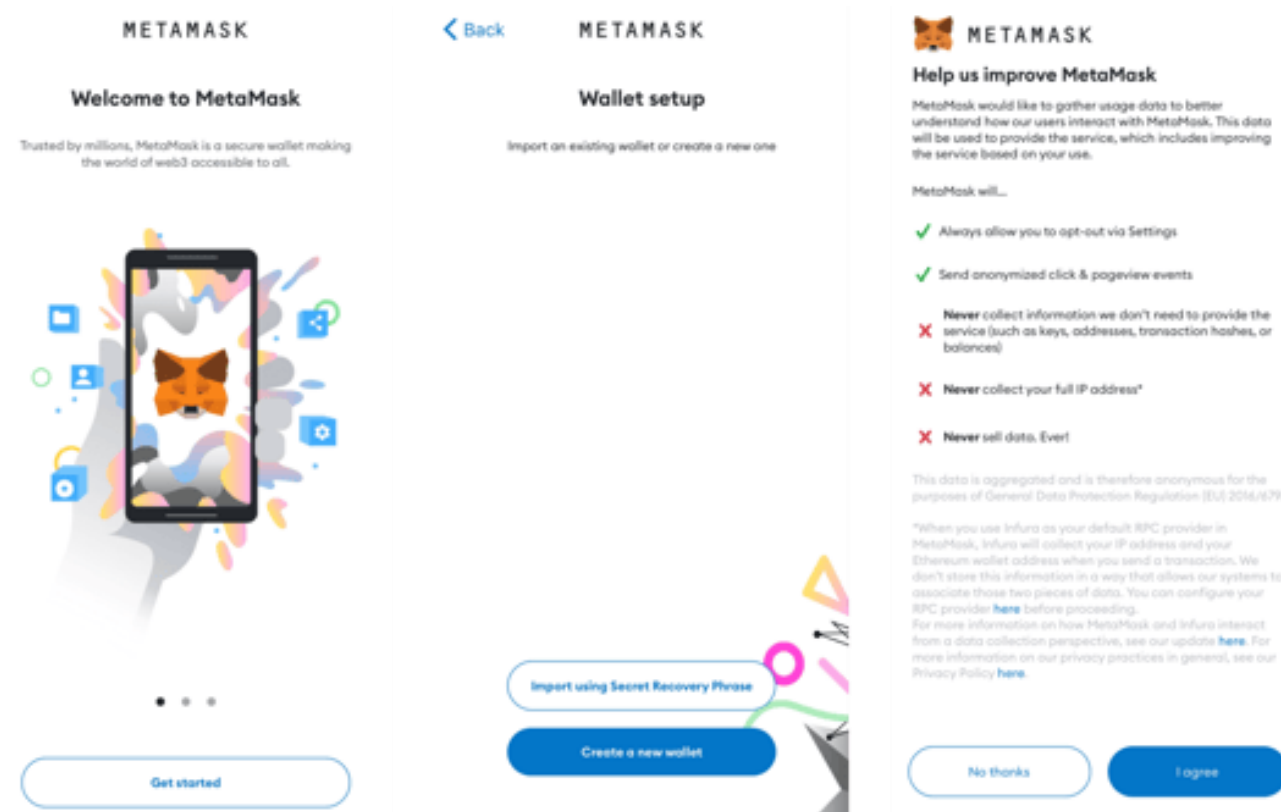
- ❖ Manages private keys
- ❖ Signs transactions
- ❖ Connects browser to blockchain

Step 1: Install MetaMask

- ❖ Download from official site:
<https://metamask.io/download>
- ❖ Click **Add to Chrome** (or your browser)

Step 2: Create a MetaMask Account

- ❖ Open MetaMask extension.
- ❖ Click **Create a Wallet**
- ❖ Set a strong password.



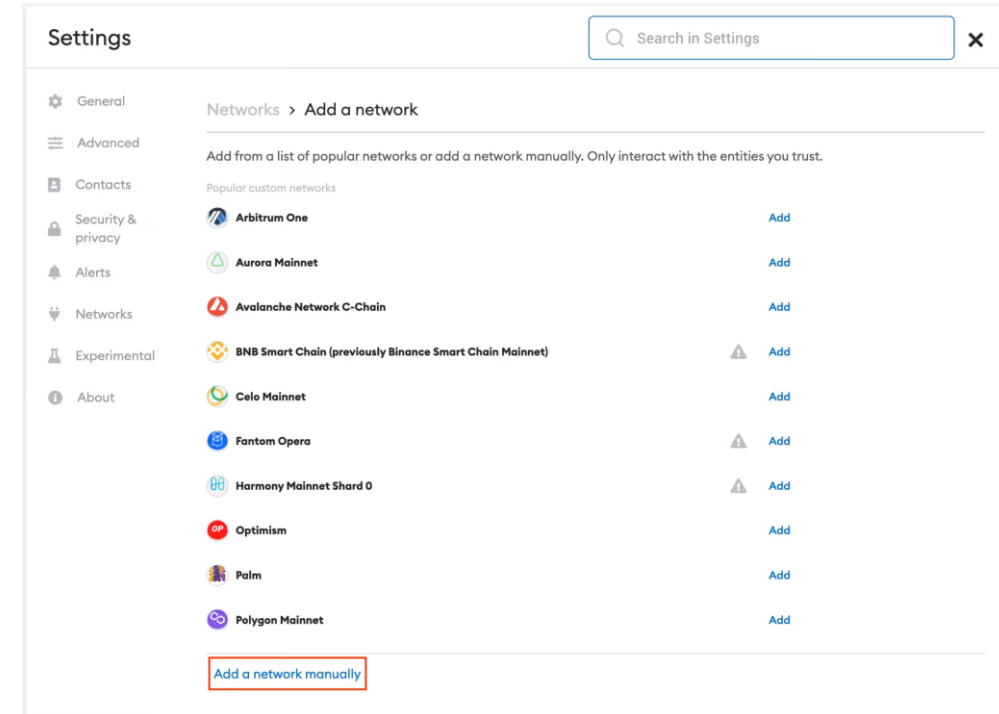
8. Introduction to Web3 Wallets & MetaMask

Step 3: Add Sepolia Testnet to MetaMask

Sepolia: Free Ethereum testnet, no real ETH risk

- ❖ In MetaMask, click **Network**, select **Add a custom network**
- ❖ Enter details:
 - Network Name: Sepolia
 - New RPC URL: **<https://ethereum-sepolia-rpc.publicnode.com>**
 - Chain ID: 11155111
 - Currency Symbol: ETH
 - Block Explorer: <https://eth-sepolia.blockscout.com/> or <https://sepolia.etherscan.io>
- ❖ Save.

Note: Currently, Sepolia is a built-in testnet in MetaMask, no import required (we can skip step 3).



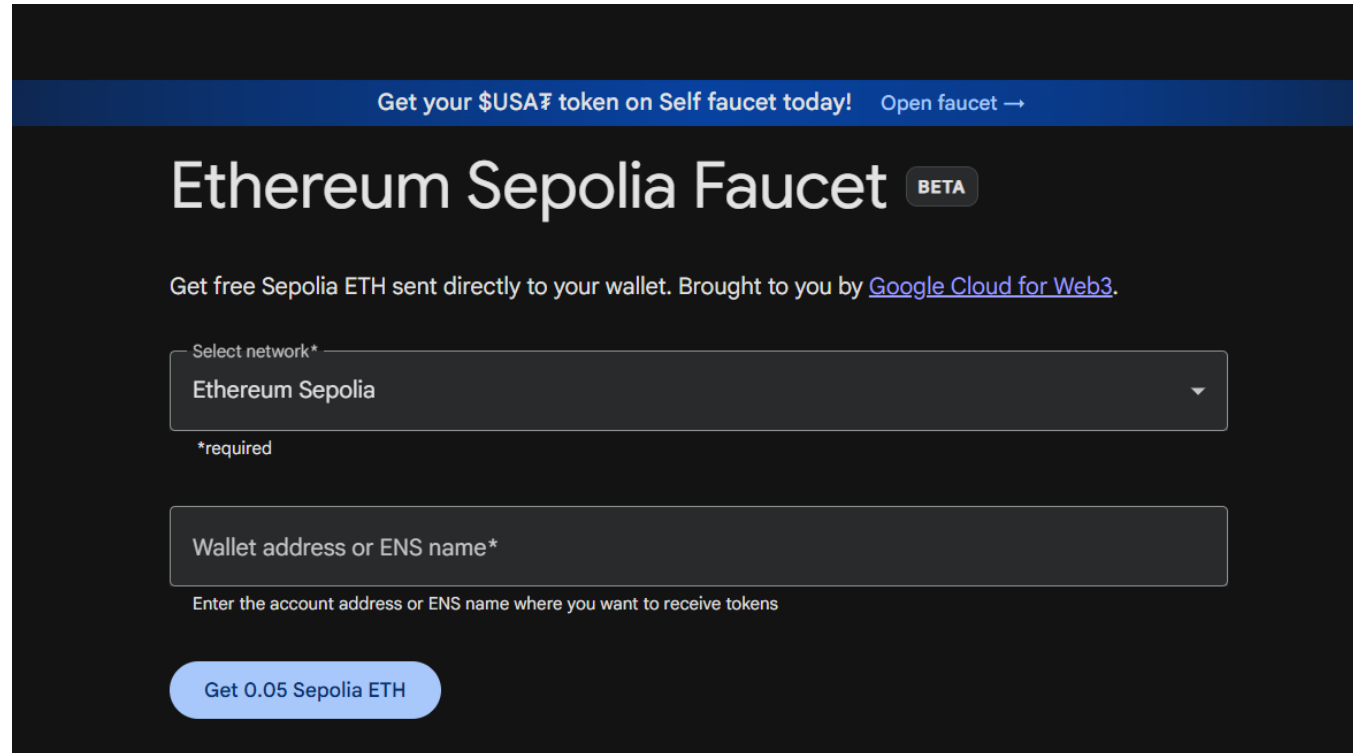
8. Introduction to Web3 Wallets & MetaMask

Step 4: Get Test ETH from a Faucet

- ❖ Go to a Sepolia faucet:

<https://cloud.google.com/application/web3/faucet/ethereum/sepolia>

- ❖ Enter your wallet address
- ❖ Request ETH



The screenshot shows the 'Ethereum Sepolia Faucet' interface. At the top, a blue banner says 'Get your \$USA₿ token on Self faucet today! Open faucet →'. Below this, the title 'Ethereum Sepolia Faucet' is displayed with a 'BETA' badge. A subtitle reads 'Get free Sepolia ETH sent directly to your wallet. Brought to you by [Google Cloud for Web3](#).' The form has two main input fields: 'Select network*' with a dropdown menu currently showing 'Ethereum Sepolia', and 'Wallet address or ENS name*'. Below the first field is a '*required' label. Below the second field is a hint: 'Enter the account address or ENS name where you want to receive tokens'. At the bottom is a blue button labeled 'Get 0.05 Sepolia ETH'.

8. Introduction to Web3 Wallets & MetaMask



Lab Practice 1: View your Sepolia ETH balance on MetaMask

- ❖ Open your MetaMask browser extension, Click the **Network** option, toggle the **Show test networks** switch to ON (if it is not already enabled).
- ❖ In the **Tokens** screen, locate the network list, scroll down to find and select the Sepolia network. Your MetaMask home screen will display your current balance in Sepolia ETH

The image shows three sequential screenshots of the MetaMask browser extension interface, connected by large blue arrows indicating the flow of the lab practice.

Screenshot 1 (Left): The MetaMask home screen. The 'Networks' option in the left sidebar is highlighted with a grey background.

Screenshot 2 (Middle): The 'Manage networks' screen. The 'Show test networks' toggle switch is turned ON (blue).

Screenshot 3 (Right): The 'Tokens' screen. The 'Network: Sepolia' is selected at the top. Below, the 'SepoliaETH' token is listed with a balance of '0.0938 SepoliaETH'.

8. Introduction to Web3 Wallets & MetaMask

Lab Practice 2: Auditing Your Wallet on Blockchain explorer

- ❖ Open your MetaMask browser extension, click on your account name to copy your Wallet Address on Sepolia (Ethereum)
- ❖ Go to the Sepolia Blockchain Explorer URL: <https://eth-sepolia.blockscout.com/>, paste your address into the search bar and press Enter.

Locate the incoming transaction from the faucet and write down:

- 1) The Transaction Hash (Tx Hash)
- 2) The Block Number where your transaction was finalized
- 3) The Gas Fee paid to the validator

